第 2 讲：CSP-S 模考 2

学而思 CSP-S 复赛模考讲评课

倪理涵

学而思素养中心

2026 年 10 月 18 日

题目大意

- 给定长度为 n, m 的字符串 s, t。
- 从 s 中选出子序列 $s_{p_1} s_{p_2} \cdots s_{p_m}$，要求与 t 至多有一个位置不同。
- 求子序列的字典序最小的下标序列，或判断无解。
- 参考难度：普及 +/提高-。

问题分析

- 理解：最多修改一次，从 s 中选出子序列 t。

问题分析

- 理解：最多修改一次，从 s 中选出子序列 t。
- 注意到测试点 $1 \sim 4$：$n, m \leq 100$。

问题分析

- 理解：最多修改一次，从 s 中选出子序列 t。
- 注意到测试点 $1 \sim 4$：$n, m \leq 100$。
  - 枚举修改、配对的位置 s_i, t_j；
  - 从 $s_{1\sim i-1}$ 中选出 $t_{1\sim j-1}$，$s_{i+1\sim n}$ 中选出 $t_{j+1\sim m}$，并要求下标序列的字典序最小。

问题分析

- 理解：最多修改一次，从 s 中选出子序列 t。
- 注意到测试点 $1 \sim 4$：$n, m \leq 100$。
  - 枚举修改、配对的位置 s_i, t_j；
  - 从 $s_{1\sim i-1}$ 中选出 $t_{1\sim j-1}$，$s_{i+1\sim n}$ 中选出 $t_{j+1\sim m}$，并要求下标序列的字典序最小。
- 贪心：依次遍历 s_i，能与 t_j 配对，则直接选出。
  - 可使用双序列双指针，维护配对过程。

问题分析

- 理解：最多修改一次，从 s 中选出子序列 t。
- 注意到测试点 $1 \sim 4$：$n, m \leq 100$。
  - 枚举修改、配对的位置 s_i, t_j；
  - 从 $s_{1\sim i-1}$ 中选出 $t_{1\sim j-1}$，$s_{i+1\sim n}$ 中选出 $t_{j+1\sim m}$，并要求下标序列的字典序最小。
- 贪心：依次遍历 s_i，能与 t_j 配对，则直接选出。
  - 可使用双序列双指针，维护配对过程。
- 注意：应考虑不修改的情形，t 直接为 s 的子序列。

问题分析

- 理解：最多修改一次，从 s 中选出子序列 t。
- 注意到测试点 $1\sim4$：$n,m\le100$。
  - 枚举修改、配对的位置 s_i,t_j；
  - 从 $s_{1\sim i-1}$ 中选出 $t_{1\sim j-1}$，$s_{i+1\sim n}$ 中选出 $t_{j+1\sim m}$，并要求下标序列的字典序最小。
- 贪心：依次遍历 s_i，能与 t_j 配对，则直接选出。
  - 可使用双序列双指针，维护配对过程。
- 注意：应考虑不修改的情形，t 直接为 s 的子序列。
- 时间复杂度 $\mathcal{O}(n^2m)$，期望得分 20 分。

问题分析

- 注意到测试点 $5 \sim 12$：$n, m \leq 2000$。

问题分析

- 注意到测试点 $5 \sim 12$：$n, m \leq 2000$。
  - 考虑只枚举其中一个字符串的修改位置。

问题分析

- 注意到测试点 $5 \sim 12$：$n, m \leq 2000$。
  - 考虑只枚举其中一个字符串的修改位置。
- 枚举 $j = 1, 2, \ldots, m$，允许 $s_{p_j} \neq t_j$。

问题分析

- 注意到测试点 $5 \sim 12$：$n, m \leq 2000$。
  - 考虑只枚举其中一个字符串的修改位置。
- 枚举 $j = 1, 2, \ldots, m$，允许 $s_{p_j} \neq t_j$。
- 固定 j 后，从左到右扫描 s：
  - 先贪心选出 $t_{1 \sim j-1}$；

问题分析

- 注意到测试点 $5 \sim 12$：$n, m \leq 2000$。
  - 考虑只枚举其中一个字符串的修改位置。
- 枚举 $j = 1, 2, \ldots, m$，允许 $s_{p_j} \neq t_j$。
- 固定 j 后，从左到右扫描 s：
  - 先贪心选出 $t_{1 \sim j-1}$；
  - 直接选出 $s_{p_{j-1}+1}$ 充当 t_j，涵盖不修改的情形；

问题分析

- 注意到测试点 $5 \sim 12$：$n, m \leq 2000$。
  - 考虑只枚举其中一个字符串的修改位置。
- 枚举 $j = 1, 2, \ldots, m$，允许 $s_{p_j} \neq t_j$。
- 固定 j 后，从左到右扫描 s：
  - 先贪心选出 $t_{1 \sim j-1}$；
  - 直接选出 $s_{p_{j-1}+1}$ 充当 t_j，涵盖不修改的情形；
  - 再贪心选出 $t_{j+1 \sim m}$。

问题分析

- 注意到测试点 $5 \sim 12$：$n, m \leq 2000$。
  - 考虑只枚举其中一个字符串的修改位置。
- 枚举 $j = 1, 2, \ldots, m$，允许 $s_{p_j} \neq t_j$。
- 固定 j 后，从左到右扫描 s：
  - 先贪心选出 $t_{1 \sim j-1}$；
  - 直接选出 $s_{p_{j-1}+1}$ 充当 t_j，涵盖不修改的情形；
  - 再贪心选出 $t_{j+1 \sim m}$。
- 时间复杂度 $\mathcal{O}(nm)$，期望得分 60 分。

问题分析

- 考虑省略修改位置的枚举，保留配对过程。

问题分析

- 考虑省略修改位置的枚举，保留配对过程。
- 遍历 s，若 $s_i = t_j$，则直接配对。

问题分析

- 考虑省略修改位置的枚举，保留配对过程。
- 遍历 s，若 $s_i = t_j$，则直接配对。
- 问题：$s_i \neq t_j$ 时，应如何操作？

问题分析

- 考虑省略修改位置的枚举，保留配对过程。
- 遍历 s，若 $s_i = t_j$，则直接配对。
- 问题：$s_i \neq t_j$ 时，应如何操作？
  - 应判断 $s_{i+1\sim n}$ 中能否选出 $t_{j+1\sim m}$；
  - 若能选出，则一定修改，才能保证字典序最小。

问题分析

- 考虑省略修改位置的枚举，保留配对过程。
- 遍历 s，若 $s_i = t_j$，则直接配对。
- 问题：$s_i \neq t_j$ 时，应如何操作？
  - 应判断 $s_{i+1\sim n}$ 中能否选出 $t_{j+1\sim m}$；
  - 若能选出，则一定修改，才能保证字典序最小。
- 可按倒序配对，预处理后缀信息。
  - $suf[i]$：$s_{i\sim n}$ 能选出的的最长 $t_{j\sim m}$ 的起始下标 j；

问题分析

- 考虑省略修改位置的枚举，保留配对过程。
- 遍历 s，若 $s_i = t_j$，则直接配对。
- 问题：$s_i \neq t_j$ 时，应如何操作？
  - 应判断 $s_{i+1\sim n}$ 中能否选出 $t_{j+1\sim m}$；
  - 若能选出，则一定修改，才能保证字典序最小。
- 可按倒序配对，预处理后缀信息。
  - $suf[i]$：$s_{i\sim n}$ 能选出的的最长 $t_{j\sim m}$ 的起始下标 j；
  - $s_{i+1\sim n}$ 中能否选出 $t_{j+1\sim m}$，等价于：

$$suf[i+1] \leq j+1$$

问题分析

- 配对时，最多修改一次，应记录修改标记：

```
1  bool changed = false; int j = 1;
2  for (int i = 1; i <= n && j <= m; ++i)
3    if (s[i] == t[j] || !changed && suf[i + 1] <= j + 1) {
4      if (s[i] != t[j]) changed = true;
5      ans[j++] = i;
6    }
```

问题分析

- 配对时，最多修改一次，应记录修改标记：

```
1  bool changed = false; int j = 1;
2  for (int i = 1; i <= n && j <= m; ++i)
3    if (s[i] == t[j] || !changed && suf[i + 1] <= j + 1) {
4      if (s[i] != t[j]) changed = true;
5      ans[j++] = i;
6    }
```

- 倒序配对，预处理 $suf[i]$：

```
1  suf[n + 1] = m + 1; // 无配对的部分
2  for (int i = n, j = m; i >= 1; --i) {
3    if (j >= 1 && s[i] == t[j]) --j;
4    suf[i] = j + 1;
5  }
```

问题分析

- 配对时，最多修改一次，应记录修改标记：

```
1  bool changed = false; int j = 1;
2  for (int i = 1; i <= n && j <= m; ++i)
3    if (s[i] == t[j] || !changed && suf[i + 1] <= j + 1) {
4      if (s[i] != t[j]) changed = true;
5      ans[j++] = i;
6    }
```

- 倒序配对，预处理 $suf[i]$：

```
1  suf[n + 1] = m + 1; // 无配对的部分
2  for (int i = n, j = m; i >= 1; --i) {
3    if (j >= 1 && s[i] == t[j]) --j;
4    suf[i] = j + 1;
5  }
```

- 注意：$suf[i+1]$ 等只判断可行性，不表示最优解。

问题分析

- 配对时，最多修改一次，应记录修改标记：

```
1  bool changed = false; int j = 1;
2  for (int i = 1; i <= n && j <= m; ++i)
3    if (s[i] == t[j] || !changed && suf[i + 1] <= j + 1) {
4      if (s[i] != t[j]) changed = true;
5      ans[j++] = i;
6    }
```

- 倒序配对，预处理 $suf[i]$：

```
1  suf[n + 1] = m + 1; // 无配对的部分
2  for (int i = n, j = m; i >= 1; --i) {
3    if (j >= 1 && s[i] == t[j]) --j;
4    suf[i] = j + 1;
5  }
```

- 注意：$suf[i+1]$ 等只判断可行性，不表示最优解。
- 时间复杂度 $\mathcal{O}(n+m)$，期望得分 100 分。



问题分析

倪理涵

T1, T2
截取密信
谁能下到 100 层
T3, T4
站岗
种萝卜

题目大意

- 有 n 个平台，高度 h_i。
- 若 $|i-j| \le d, h_i > h_j$，且 $h_{i+1}, \ldots, h_{j-1} < h_i$，则可从平台 i 可以跳到平台 j。
- 求从任意一个平台出发，最多可经过的平台个数。
- 参考难度：提高。



问题分析

- 注意到测试点 $1 \sim 4$：$n, d \leq 10$。

问题分析

- 注意到测试点 $1 \sim 4$：$n, d \leq 10$。
  - 考虑搜索起点和中间的路线。

问题分析

- 注意到测试点 $1 \sim 4$：$n, d \leq 10$。
  - 考虑搜索起点和中间的路线。
- DFS(i)：从 h_i 出发，最多可经过的平台数。

问题分析

- 注意到测试点 $1 \sim 4$：$n, d \leq 10$。
  - 考虑搜索起点和中间的路线。
- DFS(i)：从 h_i 出发，最多可经过的平台数。
- 向左跳，依次枚举 $j = i-1, \ldots, \max(i-d, 0)$。
  - 若 $h_j \geq h_i$，则终止枚举，以满足条件；
  - 同理，可搜索向右跳的情形：

```
1  for (int j = i - 1; j >= max(1, i - d) && h[j] < h[i]; --j
2    res = max(res, dfs(j) + 1);
```

问题分析

- 注意到测试点 $1 \sim 4$：$n, d \leq 10$。
  - 考虑搜索起点和中间的路线。
- DFS(i)：从 h_i 出发，最多可经过的平台数。
- 向左跳，依次枚举 $j = i - 1, \ldots, \max(i - d, 0)$。
  - 若 $h_j \geq h_i$，则终止枚举，以满足条件；
  - 同理，可搜索向右跳的情形：

```
1  for (int j = i - 1; j >= max(1, i - d) && h[j] < h[i]; --j
2    res = max(res, dfs(j) + 1);
```

- 理解：平台 i 能跳到平台 j 时，连边 $i \to j$。
  - 以上过程会搜索图中的所有路径。

问题分析

- 注意到测试点 $1 \sim 4$：$n, d \leq 10$。
  - 考虑搜索起点和中间的路线。
- DFS(i)：从 h_i 出发，最多可经过的平台数。
- 向左跳，依次枚举 $j = i-1, \ldots, \max(i-d, 0)$。
  - 若 $h_j \geq h_i$，则终止枚举，以满足条件；
  - 同理，可搜索向右跳的情形：

```
1  for (int j = i - 1; j >= max(1, i - d) && h[j] < h[i]; --j
2    res = max(res, dfs(j) + 1);
```

- 理解：平台 i 能跳到平台 j 时，连边 $i \to j$。
  - 以上过程会搜索图中的所有路径。
- 时间复杂度为 n 的指数级，期望得分 16 分。

问题分析

- 注意到测试点 $5 \sim 12$：$n, d \leq 10^3$。
  - 从同一个平台出发，后续可经过的路线和最优解都固定不变；
  - 考虑改造搜索，实现记忆化搜索。

问题分析

- 注意到测试点 $5 \sim 12$：$n, d \leq 10^3$。
  - 从同一个平台出发，后续可经过的路线和最优解都固定不变；
  - 考虑改造搜索，实现记忆化搜索。
- 记录搜索结果，若曾经搜过，则直接返回：

```
1 int &res = dp[i];
2 if (res) return res;
```

问题分析

- 注意到测试点 $5 \sim 12$：$n, d \leq 10^3$。
  - 从同一个平台出发，后续可经过的路线和最优解都固定不变；
  - 考虑改造搜索，实现记忆化搜索。
- 记录搜索结果，若曾经搜过，则直接返回：

```
1 int &res = dp[i];
2 if (res) return res;
```

- 时间复杂度 $\mathcal{O}(nd)$，期望得分 48 分。

问题分析

- 注意到特殊性质 A：$h_1 > h_2 > \cdots > h_n$。

问题分析

- 注意到特殊性质 A：$h_1 > h_2 > \cdots > h_n$。
  - 此时，只能从左向右单向跳跃。

问题分析

- 注意到特殊性质 A：$h_1 > h_2 > \cdots > h_n$。
  - 此时，只能从左向右单向跳跃。
- 状态转移方程：

$$dp[i] = \max(dp[j]) + 1, \quad i + 1 \le j \le \min(n, i + d)$$

问题分析

- 注意到特殊性质 A：$h_1 > h_2 > \cdots > h_n$。
  - 此时，只能从左向右单向跳跃。
- 状态转移方程：

$$dp[i] = \max(dp[j]) + 1, \quad i + 1 \le j \le \min(n, i + d)$$

- 倒序转移，单调队列维护 $[i+1, i+d]$ 的 $max(dp[j])$。

问题分析

- 注意到特殊性质 A：$h_1 > h_2 > \cdots > h_n$。
  - 此时，只能从左向右单向跳跃。
- 状态转移方程：

$$dp[i] = \max(dp[j]) + 1, \quad i + 1 \le j \le \min(n, i + d)$$

- 倒序转移，单调队列维护 $[i+1, i+d]$ 的 $max(dp[j])$。
- 时间复杂度 $\mathcal{O}(n)$，结合已有部分分，期望得分 60 分。

问题分析

- 逆向思维，倒着跳：

问题分析

- 逆向思维，倒着跳：
- 贪心：倒着跳时，跳到两侧最近的更大元素更优。

问题分析

- 逆向思维，倒着跳：
- 贪心：倒着跳时，跳到两侧最近的更大元素更优。
  - 可使用单调栈，维护每个元素左、右侧最近的更大元素位置。
  - 对应 DP 的转移从 $\mathcal{O}(d)$ 优化至 $\mathcal{O}(1)$。

问题分析

- 逆向思维，倒着跳：
- 贪心：倒着跳时，跳到两侧最近的更大元素更优。
  - 可使用单调栈，维护每个元素左、右侧最近的更大元素位置。
  - 对应 DP 的转移从 $\mathcal{O}(d)$ 优化至 $\mathcal{O}(1)$。
- 时间复杂度 $\mathcal{O}(n)$，期望得分 100 分。

题目大意

- 给定边长为 a 的正方形，及其上的 n 岗哨位。
- 要求从中选出恰好 k 个位置。
- 求任意两点之间的曼哈顿距离的最小值的最大值。
- 参考难度：提高。

问题分析

- 按顺时针方向，将正方形边界上的点映射到长度为 $4a$ 的环上：
  - 左边界 $x = 0$：$p = y$；
  - 上边界 $y = a$：$p = a + x$；
  - 右边界 $x = a$：$p = 3a - y$；
  - 下边界 $y = 0$：$p = 4a - x$。

问题分析

- 按顺时针方向，将正方形边界上的点映射到长度为 $4a$ 的环上：
  - 左边界 $x = 0$：$p = y$；
  - 上边界 $y = a$：$p = a + x$；
  - 右边界 $x = a$：$p = 3a - y$；
  - 下边界 $y = 0$：$p = 4a - x$。
- 对于边界上的两点，曼哈顿距离等于它们在环上的较短弧长。

问题分析

- 按顺时针方向，将正方形边界上的点映射到长度为 $4a$ 的环上：
  - 左边界 $x = 0$：$p = y$；
  - 上边界 $y = a$：$p = a + x$；
  - 右边界 $x = a$：$p = 3a - y$；
  - 下边界 $y = 0$：$p = 4a - x$。
- 对于边界上的两点，曼哈顿距离等于它们在环上的较短弧长。
- 因此，将选出的点按环上顺序排列后，只需保证相邻两个选中点的环上距离都至少为 low。
- 二分最大的 low，问题变成：能否在环上选出 k 个点，使相邻距离都不少于 low？

问题分析

- 设选中的点在环上依次为 $p_1, p_2, \cdots, p_k$。
- 相邻的 $k-1$ 个间隔需要满足

$$p_{i+1} - p_i \geq low.$$

- 首尾之间还需要满足环形距离条件

$$4a - (p_k - p_1) \geq low,$$

即 $p_k - p_1 \leq 4a - low$.

问题分析

- 设选中的点在环上依次为 $p_1, p_2, \cdots, p_k$。
- 相邻的 $k-1$ 个间隔需要满足

$$p_{i+1} - p_i \geq low.$$

- 首尾之间还需要满足环形距离条件

$$4a - (p_k - p_1) \geq low,$$

即 $p_k - p_1 \leq 4a - low$.

- 由于 $k \geq 4$，答案不会超过正方形边长 a，所以沿边界顺序考虑相邻点即可。
- 特别地，low 越大，满足条件的方案越少，具有单调性，可以二分答案。

问题分析

- 注意到测试点 $1 \sim 2$：$n \leq 15$。

问题分析

- 注意到测试点 $1 \sim 2$：$n \leq 15$。
  - 枚举选出的 k 个点；
  - 求 k 个点中所有点对的曼哈顿距离，取最小值。
- 再对所有选法取最大值。

问题分析

- 注意到测试点 $1 \sim 2$：$n \leq 15$。
  - 枚举选出的 k 个点；
  - 求 k 个点中所有点对的曼哈顿距离，取最小值。
- 再对所有选法取最大值。
- 复杂度约为

  ，
- 时间复杂度 $\mathcal{O}(k^2 C_n^k)$，期望得分 10 分。

问题分析

- 先固定 low，枚举环上的第一个选中点 p_s。
- 从 p_s 开始，不断二分查找下一个满足

$$p_j \geq p_{j-1} + low$$

的最靠左位置。

- 找到 k 个点后，检查首尾是否满足

$$p_k - p_s \leq 4a - low.$$

问题分析

- 先固定 low，枚举环上的第一个选中点 p_s。
- 从 p_s 开始，不断二分查找下一个满足

$$p_j \geq p_{j-1} + low$$

的最靠左位置。

- 找到 k 个点后，检查首尾是否满足

$$p_k - p_s \leq 4a - low.$$

- 若存在一个起点可以选出合法方案，则 low 可行。
- 这是“在一条链上尽量早地选点”的贪心：越早选，后面留下的空间越大。

问题分析

- 每个起点最多向后找 $k-1$ 个点，每次用二分查找下一个位置。
- 单次 `check(low)` 的复杂度为

$$\mathcal{O}(nk\log n),$$

- 再套一层答案二分，总复杂度为

$$\mathcal{O}(nk\log n\log a).$$

- 当 $n\le 1000$ 时可以通过，期望得分 30 分。

问题分析

- 每个起点最多向后找 $k-1$ 个点，每次用二分查找下一个位置。
- 单次 `check(low)` 的复杂度为

$$\mathcal{O}(nk\log n),$$

- 再套一层答案二分，总复杂度为

$$\mathcal{O}(nk\log n\log a).$$

- 当 $n\le 1000$ 时可以通过，期望得分 30 分。
- 代码实现时，点已经按环上坐标排序；不需要复制一份数组，只需枚举起点并检查首尾距离。

问题分析

- 方法一的瓶颈在于：每找一个下一个点，都重新进行一次二分查找。
- 维护 k 个指针 idx[0],idx[1],…,idx[k-1]，表示当前起点下选择的 k 个点。

问题分析

- 方法一的瓶颈在于：每找一个下一个点，都重新进行一次二分查找。
- 维护 k 个指针 idx[0],idx[1],…,idx[k-1]，表示当前起点下选择的 k 个点。
- 依次调整 $j = 1, 2, \ldots, k-1$：

```
1 while (p[idx[j]] < p[idx[j-1]] + low)
2     ++idx[j];
```

- 所有指针只会向右移动；起点 idx[0] 增加后继续维护。

问题分析

- 方法一的瓶颈在于：每找一个下一个点，都重新进行一次二分查找。
- 维护 k 个指针 idx[0],idx[1],…,idx[k-1]，表示当前起点下选择的 k 个点。
- 依次调整 $j = 1, 2, \ldots, k-1$:

```
1 while (p[idx[j]] < p[idx[j-1]] + low)
2     ++idx[j];
```

- 所有指针只会向右移动；起点 idx[0] 增加后继续维护。
- 找到 k 个点后检查

$$p_{idx[k-1]} - p_{idx[0]} \leq 4a - low.$$

问题分析

- 对固定的 low，每个指针整体最多向右移动 n 次。
- 枚举起点并维护 k 个指针，单次检查复杂度为

$$\mathcal{O}(nk).$$

- 答案二分后的总复杂度为

$$\mathcal{O}(nk \log a).$$

- 当 $k \leq 25$ 时，nk 足够小，可以通过测试点 $7 \sim 12$，期望得分 60 分。

问题分析

- 对固定的 low，每个指针整体最多向右移动 n 次。
- 枚举起点并维护 k 个指针，单次检查复杂度为

$$\mathcal{O}(nk).$$

- 答案二分后的总复杂度为

$$\mathcal{O}(nk \log a).$$

- 当 $k \leq 25$ 时，nk 足够小，可以通过测试点 $7 \sim 12$，期望得分 60 分。
- 方法三“二分答案 + 倍增”只是继续优化找点过程。这里不展开，直接观察完整数据下的结构。

问题分析

- 性质 A：所有点在同一条边上。
  - 环上的点集中在长度为 a 的一段中；
  - 因为 $k \geq 4$，可行的 $low \leq a$，首尾绕环的距离自动满足要求；
  - 问题退化为一条链上的选点，线性贪心即可。

问题分析

- 性质 A：所有点在同一条边上。
  - 环上的点集中在长度为 a 的一段中；
  - 因为 $k \geq 4$，可行的 $low \leq a$，首尾绕环的距离自动满足要求；
  - 问题退化为一条链上的选点，线性贪心即可。
- 性质 B：$k = n$。
  - 所有点都必须选；
  - 答案就是排序后相邻点距离以及首尾环形距离的最小值。

问题分析

- 性质 A：所有点在同一条边上。
  - 环上的点集中在长度为 a 的一段中；
  - 因为 $k \geq 4$，可行的 $low \leq a$，首尾绕环的距离自动满足要求；
  - 问题退化为一条链上的选点，线性贪心即可。
- 性质 B：$k = n$。
  - 所有点都必须选；
  - 答案就是排序后相邻点距离以及首尾环形距离的最小值。
- 结合测试点范围可以拿到 80 分，但完整数据仍需要处理环形起点的选择。

问题分析

- 仍然二分 low，先将环上坐标排序为

$$p_0 < p_1 < \cdots < p_{n-1}.$$

- 定义 nxt[i]：从 p_i 出发，向后距离至少为 low 的第一个位置。
- 用双指针在线性时间内求出所有 nxt[i]：

```
1  int j = n;
2  for (int i = n - 1; i >= 0; --i) {
3      while (j - 1 >= 0 && p[j - 1] >= p[i] + low) --j;
4      nxt[i] = j;
5  }
```

- 不存在这样的点时令 nxt[i]=n，其中 n 是哨兵。

问题分析

- 对每个位置 i，加边 nxt[i] $\longrightarrow$ i。
- nxt[i]>i，故每个点最终都会连到哨兵 n，得到根为 n 的有向树。

问题分析

- 对每个位置 i，加边 nxt[i] $\longrightarrow$ i。
- nxt[i]>i，故每个点最终都会连到哨兵 n，得到根为 n 的有向树。
- 树上一条从根向下的路径，表示一串位置：相邻两个位置的距离都至少为 low。
- 问题即求：是否存在一条根到某个节点的路径，其中包含至少 k 个真实位置，并且首尾满足环形距离条件？

问题分析

- 对每个位置 i，加边 nxt[i] $\longrightarrow$ i。
- nxt[i]>i，故每个点最终都会连到哨兵 n，得到根为 n 的有向树。
- 树上一条从根向下的路径，表示一串位置：相邻两个位置的距离都至少为 low。
- 问题即求：是否存在一条根到某个节点的路径，其中包含至少 k 个真实位置，并且首尾满足环形距离条件？
- 无需枚举起点，用一次 DFS 同时检查所有起点。

问题分析

- DFS 时，用栈记录从根到当前节点的路径位置。
- 若路径中已有 k 个真实点，设这 k 个点的第一个和最后一个坐标为 x, y，则需要 $y - x \leq 4a - low$。
  - 首尾绕环的距离为 $4a - (y - x)$，上式等价于首尾距离 $\geq low$。

问题分析

- DFS 时，用栈记录从根到当前节点的路径位置。
- 若路径中已有 k 个真实点，设这 k 个点的第一个和最后一个坐标为 x, y，则需要 $y - x \leq 4a - low$。
  - 首尾绕环的距离为 $4a - (y - x)$，上式等价于首尾距离 $\geq low$。
- 对应的判断：

```
1  if ((int)path.size() > k &&
2      path[path.size() - k] - p[x] <= 4LL * a - low)
3      return true;
```

- 若检查完所有子树仍未找到，则 low 不可行。

问题分析

- nxt[i] 保留了从 p_i 出发能连接到的最靠前位置。
- 所有满足相邻距离至少为 low 的选点序列，都可以对应到树上的一条路径；DFS 会遍历所有可能的路径。
- 路径上的相邻点已经保证间距至少为 low，再检查首尾绕环距离即可判断方案是否合法。

问题分析

- nxt[i] 保留了从 p_i 出发能连接到的最靠前位置。
- 所有满足相邻距离至少为 low 的选点序列，都可以对应到树上的一条路径；DFS 会遍历所有可能的路径。
- 路径上的相邻点已经保证间距至少为 low，再检查首尾绕环距离即可判断方案是否合法。
- 时间复杂度 $\mathcal{O}(n \log a)$，期望得分 100 分。

题目大意

- 第 j 列萝卜的种植深度记为 h_j，其中 $0 \le h_j \le n$。
- 第 j 列前 h_j 行被挖去，剩余位置没有萝卜。
- 对于没有被挖去的格子，如果左右相邻位置至少有一个被挖去，则该格子的养分会被吸取。
- 求所有种植方案中，能够吸取的养分总和的最大值。
- 参考难度：省选 +/NOI-。

问题分析

CSP-S 模考 2

倪理涵

T1, T2
截取密信
谁能下到 100 层

T3, T4
站岗
种萝卜

- 一列中被挖去的格子一定是从第 1 行开始的连续前缀，因此一个方案可以唯一表示为

$$h_1, h_2, \ldots, h_n, \qquad 0 \le h_j \le n$$

问题分析

CSP-S 模考 2

倪理涵

T1, T2
截取密信
谁能下到 100 层

T3, T4
站岗
种萝卜

- 一列中被挖去的格子一定是从第 1 行开始的连续前缀，因此一个方案可以唯一表示为

$$h_1, h_2, \ldots, h_n, \qquad 0 \le h_j \le n$$

- 约定边界外两列的高度为 $h_0 = h_{n+1} = 0$。

问题分析

- 一列中被挖去的格子一定是从第 1 行开始的连续前缀，因此一个方案可以唯一表示为

$$h_1, h_2, \ldots, h_n, \qquad 0 \le h_j \le n$$

- 约定边界外两列的高度为 $h_0 = h_{n+1} = 0$。
- $h_j \le i < \max(h_{j-1}, h_{j+1})$ 时，(i, j) 才计入答案。
- 故第 j 列的贡献只与相邻三列的高度有关。

问题分析

- 记 $s_j[x] = \sum_{i=0}^{x-1} a_{i,j}$，　　$s_j[0] = 0$。
- 若左边、当前、右边三列的高度分别为 x, y, z，则当前列贡献为

$$w_j(x, y, z) = \max(0, s_j[\max(x, z)] - s_j[y]).$$

问题分析

- 记 $s_j[x] = \sum_{i=0}^{x-1} a_{i,j}$，$s_j[0] = 0$。
- 若左边、当前、右边三列的高度分别为 x, y, z，则当前列贡献为

$$w_j(x, y, z) = \max(0, s_j[\max(x, z)] - s_j[y]).$$

- $\max(0, \cdot)$ 对应：只有当左右较高的萝卜超过当前萝卜时，才有未挖去且会被吸取的格子。

问题分析

- 记 $s_j[x] = \sum_{i=0}^{x-1} a_{i,j}$，$s_j[0] = 0$。
- 若左边、当前、右边三列的高度分别为 x, y, z，则当前列贡献为

$$w_j(x, y, z) = \max(0, s_j[\max(x, z)] - s_j[y]).$$

- $\max(0, \cdot)$ 对应：只有当左右较高的萝卜超过当前萝卜时，才有未挖去且会被吸取的格子。
- 预处理每一列的前缀和后，任意一个三列状态的贡献都可以在 $\mathcal{O}(1)$ 时间内算出。

问题分析

- 测试点 $1 \sim 2$ 中 $n \leq 8$。
- 枚举每一列的高度 $h_j \in [0, n]$，共有 $(n+1)^n$ 种方案。
- 对每一种方案，枚举所有格子，判断左右是否有萝卜并统计答案。

问题分析

- 测试点 $1 \sim 2$ 中 $n \leq 8$。
- 枚举每一列的高度 $h_j \in [0, n]$，共有 $(n+1)^n$ 种方案。
- 对每一种方案，枚举所有格子，判断左右是否有萝卜并统计答案。
- 时间复杂度约为

$$\mathcal{O}\big((n+1)^n \cdot n^2\big),$$

- 期望得分 8 分。

问题分析

- 处理到第 j 列时，定义

$$dp[x][y]$$

表示当前列高度为 x、前一列高度为 y 时，前面列的最优得分。

问题分析

- 处理到第 j 列时，定义

$$dp[x][y]$$

表示当前列高度为 x、前一列高度为 y 时，前面列的最优得分。

- 枚举下一列高度 z，就可以确定第 j 列的贡献：

$$dp'[z][x] \leftarrow \max(dp'[z][x], dp[x][y] + w_j(y, x, z))$$

问题分析

- 处理到第 j 列时，定义

$$dp[x][y]$$

表示当前列高度为 x、前一列高度为 y 时，前面列的最优得分。

- 枚举下一列高度 z，就可以确定第 j 列的贡献：

$$dp'[z][x] \leftarrow \max(dp'[z][x], dp[x][y] + w_j(y, x, z))$$

- 时间复杂度 $\mathcal{O}(n^4)$，期望得分 24 分。

问题分析

- 设当前处理的是第 j 列，当前高度为 x，前一列高度为 y，下一列高度为 z。
- 第 j 列新增的得分为

$$\max(0, s_j[\max(y, z)] - s_j[x])$$

问题分析

- 设当前处理的是第 j 列，当前高度为 x，前一列高度为 y，下一列高度为 z。
- 第 j 列新增的得分为

$$\max(0, s_j[\max(y,z)] - s_j[x])$$

- 转移后，下一状态的“当前列”变为第 $j+1$ 列，因此状态下标变成 (z,x)。

```
1 add = max(0LL, sum[j][max(y, z)] - sum[j][x]);
2 ndp[z][x] = max(ndp[z][x], dp[x][y] + add);
```

问题分析

- 设当前处理的是第 j 列，当前高度为 x，前一列高度为 y，下一列高度为 z。
- 第 j 列新增的得分为

$$\max(0, s_j[\max(y,z)] - s_j[x])$$

- 转移后，下一状态的“当前列”变为第 $j+1$ 列，因此状态下标变成 (z,x)。

```
1 add = max(0LL, sum[j][max(y, z)] - sum[j][x]);
2 ndp[z][x] = max(ndp[z][x], dp[x][y] + add);
```

- 处理完最后一列后，补上边界列高度 0，取所有状态的最大值。

问题分析

- 方法一中，固定 x, z 后，需要枚举前一列高度 y：

$$\max_{y} \left\{ dp[x][y] + \max(0, s_j[\max(y, z)] - s_j[x]) \right\}.$$

问题分析

- 方法一中，固定 x, z 后，需要枚举前一列高度 y：

$$\max_y \left\{ dp[x][y] + \max(0, s_j[\max(y, z)] - s_j[x]) \right\}.$$

- 按照 $y \le z$ 与 $y > z$ 分成两类：
  - $y \le z$ 时，$\max(y, z) = z$，新增部分与 y 无关，只需维护 $dp[x][y]$ 的前缀最大值；
  - $y > z$ 时，$\max(y, z) = y$，维护

$$dp[x][y] + \max(0, s_j[y] - s_j[x])$$

  的后缀最大值。

问题分析

- 方法一中，固定 x, z 后，需要枚举前一列高度 y：

$$\max_y \left\{ dp[x][y] + \max(0, s_j[\max(y, z)] - s_j[x]) \right\}.$$

- 按照 $y \le z$ 与 $y > z$ 分成两类：
  - $y \le z$ 时，$\max(y, z) = z$，新增部分与 y 无关，只需维护 $dp[x][y]$ 的前缀最大值；
  - $y > z$ 时，$\max(y, z) = y$，维护

$$dp[x][y] + \max(0, s_j[y] - s_j[x])$$

  的后缀最大值。
- 每个 (x, z) 的转移可以降为 $\mathcal{O}(1)$，每列只需处理 $\mathcal{O}(n^2)$ 个状态。

问题分析

- 对每个固定的 x，从左到右预处理前缀最大值，从右到左预处理后缀最大值。
- 状态转移从三重枚举降为两重枚举。

问题分析

- 对每个固定的 x，从左到右预处理前缀最大值，从右到左预处理后缀最大值。
- 状态转移从三重枚举降为两重枚举。
- 时间复杂度 $\mathcal{O}(n^3)$，期望得分 48 分。

问题分析

- 对每个固定的 x，从左到右预处理前缀最大值，从右到左预处理后缀最大值。
- 状态转移从三重枚举降为两重枚举。
- 时间复杂度 $\mathcal{O}(n^3)$，期望得分 48 分。
- 正解需利用“高度只通过相邻列产生影响”这一结构，将状态压缩到 $\mathcal{O}(n)$。

问题分析

- 假设所有格子的养分值都为 c。

问题分析

- 假设所有格子的养分值都为 c。
  - 若一列挖空，即 $h_j = n$，则贡献为 0。
  - 若一列不挖，即 $h_j = 0$，只要左右至少一列挖空，就能贡献 $c \cdot n$。

问题分析

- 假设所有格子的养分值都为 c。
  - 若一列挖空，即 $h_j = n$，则贡献为 0。
  - 若一列不挖，即 $h_j = 0$，只要左右至少一列挖空，就能贡献 $c \cdot n$。
- 最优解中，每列高度只需取 0 或 n。
- 问题即求：选一些互不相邻的挖空列，使被左右邻居激活的列数最多。

问题分析

- 假设所有格子的养分值都为 c。
  - 若一列挖空，即 $h_j = n$，则贡献为 0。
  - 若一列不挖，即 $h_j = 0$，只要左右至少一列挖空，就能贡献 $c \cdot n$。
- 最优解中，每列高度只需取 0 或 n。
- 问题即求：选一些互不相邻的挖空列，使被左右邻居激活的列数最多。
- 每三个连续列至多选择一个挖空列，最多激活 $\left\lfloor \frac{2n}{3} \right\rfloor$ 列，答案即为 $c \cdot n \cdot \left\lfloor \frac{2n}{3} \right\rfloor$。
- 期望得分 60 分。

问题分析

- 设第 j 列每个格子的养分值都为 c_j。
- 同样可以令每列高度只取 0 或 n：
  - 挖空列贡献 0；
  - 未挖列若被相邻挖空列激活，贡献 $n \cdot c_j$。

问题分析

- 设第 j 列每个格子的养分值都为 c_j。
- 同样可以令每列高度只取 0 或 n：
  - 挖空列贡献 0；
  - 未挖列若被相邻挖空列激活，贡献 $n \cdot c_j$。
- 于是变成带权的 $0/1$ 支配集：选择不相邻的挖空列，最大化被激活列权值之和。
- 令 $f[j][x][y]$ 表示处理到第 j 列，最后两列的挖空状态为 x, y 时的最大权值，直接枚举下一列状态即可在线性时间完成。

问题分析

- 设第 j 列每个格子的养分值都为 c_j。
- 同样可以令每列高度只取 0 或 n：
  - 挖空列贡献 0；
  - 未挖列若被相邻挖空列激活，贡献 $n \cdot c_j$。
- 于是变成带权的 $0/1$ 支配集：选择不相邻的挖空列，最大化被激活列权值之和。
- 令 $f[j][x][y]$ 表示处理到第 j 列，最后两列的挖空状态为 x, y 时的最大权值，直接枚举下一列状态即可在线性时间完成。
- 期望得分 72 分。

问题分析

- 从左向右处理列，并记录上一列高度 pre 与当前列高度 cur。

问题分析

- 从左向右处理列，并记录上一列高度 pre 与当前列高度 cur。
- 相邻两列之间新增的得分，只取决于哪一侧更高：
  - $cur > pre$：第 pre 列中高度区间 $[pre, cur)$ 被右侧萝卜激活；
  - $cur \leq pre$：第 cur 列中高度区间 $[cur, pre)$ 被左侧萝卜激活。

问题分析

- 从左向右处理列，并记录上一列高度 pre 与当前列高度 cur。
- 相邻两列之间新增的得分，只取决于哪一侧更高：
  - $cur > pre$：第 pre 列中高度区间 $[pre, cur)$ 被右侧萝卜激活；
  - $cur \leq pre$：第 cur 列中高度区间 $[cur, pre)$ 被左侧萝卜激活。
- 记录当前边界的计分归属：
  - pick：当前列已经按左侧关系计入；
  - skip：当前列的计分暂留给右侧关系处理。
- 两个状态避免一个波谷位置被左右重复计算。

问题分析

- 设上一列高度为 pre，当前列高度为 cur。
- 当 $cur > pre$ 时，前一列的区间 $[pre, cur)$ 被激活，新增

$$s_{j-1}[cur] - s_{j-1}[pre]$$

- 当 $cur \leq pre$ 时，当前列的区间 $[cur, pre)$ 被激活，新增

$$s_j[pre] - s_j[cur]$$

问题分析

- 设上一列高度为 pre，当前列高度为 cur。
- 当 $cur > pre$ 时，前一列的区间 $[pre, cur)$ 被激活，新增

$$s_{j-1}[cur] - s_{j-1}[pre]$$

- 当 $cur \leq pre$ 时，当前列的区间 $[cur, pre)$ 被激活，新增

$$s_j[pre] - s_j[cur]$$

- 因为养分值非负，不需要再写复杂的正数判断；区间端点的大小关系已经保证新增值非负。

问题分析

- 设上一列高度为 pre，当前列高度为 cur。
- 当 $cur > pre$ 时，前一列的区间 $[pre, cur)$ 被激活，新增

$$s_{j-1}[cur] - s_{j-1}[pre]$$

- 当 $cur \leq pre$ 时，当前列的区间 $[cur, pre)$ 被激活，新增

$$s_j[pre] - s_j[cur]$$

- 因为养分值非负，不需要再写复杂的正数判断；区间端点的大小关系已经保证新增值非负。
- 对于每个 cur，需要分别求前缀或后缀中的最大值：
  - $pre < cur$：使用前缀最大值；
  - $pre \geq cur$：使用后缀最大值。

问题分析

- 对 cur 固定，$pre < cur$ 的转移可以整理为

$$\max_{pre<cur} \{dp[pre][skip] - s_{j-1}[pre]\} + s_{j-1}[cur].$$

- 从左到右扫描 pre，维护一个前缀最大值即可。

问题分析

- 对 cur 固定，$pre < cur$ 的转移可以整理为

$$\max_{pre<cur} \{dp[pre][skip] - s_{j-1}[pre]\} + s_{j-1}[cur].$$

- 从左到右扫描 pre，维护一个前缀最大值即可。
- $pre \geq cur$ 的转移可以整理为

$$\max_{pre\geq cur} \{dp[pre][pick] + s_j[pre]\} - s_j[cur],$$

- 从右到左扫描 pre，维护两个后缀最大值即可。

问题分析

- 对 cur 固定，$pre < cur$ 的转移可以整理为

$$\max_{pre<cur} \{dp[pre][skip] - s_{j-1}[pre]\} + s_{j-1}[cur].$$

- 从左到右扫描 pre，维护一个前缀最大值即可。
- $pre \geq cur$ 的转移可以整理为

$$\max_{pre\geq cur} \{dp[pre][pick] + s_j[pre]\} - s_j[cur],$$

- 从右到左扫描 pre，维护两个后缀最大值即可。
- 因此，一列的所有 $cur = 0, 1, \ldots, n$ 只需线性处理。
- 时间复杂度 $\mathcal{O}(n^2)$，期望得分 100 分。

Thanks for your attention!